

Individual Project Progress Report

(June 8 – July 15)

(Name: Shefa Tabassum. Id: 2232993042)

Week 1 (Jun 8 – Jun 10)

Classes for CSE440.1 Artificial Intelligence had just started and the project team was not formed yet. I reviewed the course syllabus and the semester project requirements so I would be ready once the team and topic were decided.

Week 2 (Jun 15 – Jun 17)

The project team was formed with four members. As a group we discussed a few possible AI project ideas and settled on **Autonomous Volcanic Terrain Exploration Using a Markov Decision Process (MDP)**: an autonomous rover that must navigate a hazardous, randomly generated grid terrain (lava, craters, gas zones, rock obstacles) while balancing safety, exploration coverage, and science-point collection. We agreed an MDP was appropriate because the rover's movement is stochastic and its decisions are sequential, which fits the state/action/reward/policy structure of an MDP directly.

Week 3 (Jun 22 – Jun 24)

The team opened a shared GitHub repository and split the work into four roles so that progress could be made in parallel: MDP core (terrain generation, state/action space, reward model, value iteration), agent & simulation, visualization & demo outputs, and experiments & comparison. I was assigned **Member 3 – Visualization and Demo Outputs**. My responsibility is to turn the terrain grid and the agent's path into a clear, readable image, without modifying the MDP, terrain generation, agent, or simulation logic that the other members own.

Week 4 (Jun 29 – Jul 1)

Before writing any code, I read through the modules my part depends on so my visualization would match the real data structures instead of guessing at them:

- `support/terrain.py` – the `Terrain` class stores the grid as `terrain.grid`, a list of lists of strings, with `terrain.get_cell(row, col)` returning one of seven cell types: `SAFE`, `LAVA`, `CRATER`, `GAS`, `ROCK`, `SCIENCE`, `BASE`.
- `support/agent.py` – once finished,

`MDPExplorerAgent.path` would hold the agent's visited cells as a list of `(row, col)` tuples, which is the shape my visualization needs to accept later.

I then planned the structure of `support/visualization.py` (a terrain-rendering function, a legend builder, and a save function) and created my working branch, `member3-visualization`, following the group's Git workflow of one branch per member.

Week 5 (Jul 6 – Jul 8)

Began implementing `support/visualization.py` with `matplotlib`. I wrote `draw_terrain()`, which loops over every `(row, col)` in the grid and draws a colored `Rectangle` for each cell, and `build_legend_handles()`, which builds one legend patch per terrain type so the map is self-explanatory without reading the code. I assigned a distinct, visually separable color to each of the seven cell types:

```
CELL_COLORS = {
    SAFE: "#e8e4d8",
    LAVA: "#c0392b",
    CRATER: "#6b3fa0",
    GAS: "#8fbf6f",
    ROCK: "#5c5c5c",
    SCIENCE: "#f2c744",
    BASE: "#2f6fbf",
}
```

One detail I had to handle carefully was the coordinate system: grid row 0 is the top row when printed as text, but `matplotlib`'s y-axis increases upward, so each cell had to be plotted at `y = terrain.rows - 1 - row` to avoid drawing the terrain upside down.

Week 6 (Jul 13 – Jul 15) – Update 1 Complete

Finished and tested `support/visualization.py`:

- Added `render_terrain_map()`, which draws the terrain, adds a title and legend, and saves the figure to `outputs/final_map.png` using the non-interactive `Agg` backend so it also works without a display.
- Added `make_sample_path()` and

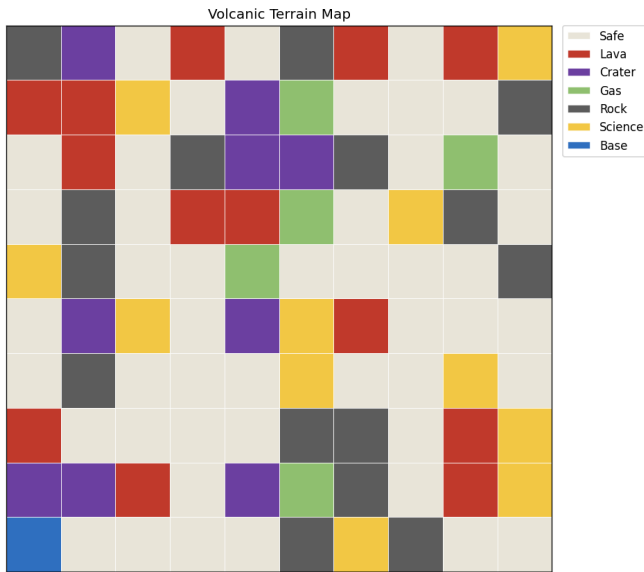


Figure 1: Terrain rendering test: all seven cell types drawn with distinct colors, a title, and a legend, before path support was added.

`draw_sample_path()` to overlay a short, clearly labeled sample path starting at BASE, since Member 2's finished agent path was not ready to plug in yet. The path is drawn as a dashed line with a star marking the start, and is explicitly labeled "sample path – test only, not a final result" in both the title and the legend, per the group rule against presenting placeholder data as a real result.

- Verified the acceptance test `python support/visualization.py` runs without errors and produces a correct, readable map (Figure 2).
- Committed and pushed the work on branch `member3-visualization`; opened a pull request and merged it into `main` after review.
- After merging, I generated three additional terrain-only sample maps at different seeds and added a "Terrain Visualization" section with example images to `README.md`; I also corrected two lines in the `README` that still described visualization as "planned for a later week," since that was no longer accurate.
- Did not modify `terrain.py`, `mdp.py`, `agent.py`, `simulation.py`, or `main.py`, per the group's file-ownership rules.

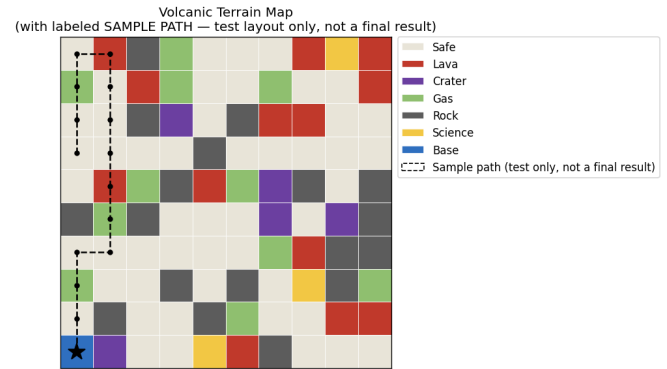


Figure 2: Final Update 1 output from `support/visualization.py`: terrain map with a clearly labeled sample path (test only, not a final result).

Testing and Verification

I ran the acceptance test directly from the project root before pushing, to confirm the module runs cleanly end to end and produces the expected file:

```
$ python support/visualization.py
Terrain size: 10x10
Sample path length (test only): 15
Saved terrain map: outputs/final_map.png
```

No exceptions were raised, and `outputs/final_map.png` was created with the correct terrain colors, legend, title, and sample path on every run.

My Personal Contribution (Summary)

Reviewed the terrain and agent data structures my module depends on before writing any code. Implemented the full terrain visualization module (`support/visualization.py`): colored grid rendering for all seven terrain types, a title and legend, and a clearly labeled sample path for layout testing. Verified the module runs standalone and produces `outputs/final_map.png`. Committed and merged the work on my own branch, and added supporting terrain-map images and documentation to the `README` – without editing any file owned by another team member.